# Swahili-English Translation System using BLOOM

A low-resource machine translation system that fine-tunes the BLOOM-560M model for Swahili-to-English translation.

## 📋 Project Overview

This project implements a complete pipeline for training and evaluating a neural machine translation system for the low-resource language pair Swahili-English. The system uses the pre-trained BLOOM (BigScience Large Open-science Open-access Multilingual) model and fine-tunes it on parallel Swahili-English data.

### Key Features

- ✅ Complete data preprocessing pipeline
- ✅ BLOOM model fine-tuning with mixed precision (FP16)
- ✅ BLEU score evaluation
- ✅ Interactive inference mode
- ✅ Modular, well-documented code
- ✅ GPU acceleration support

## 🗂️ Project Structure

```
swahili-translation/
│
├── config.py              # Configuration and hyperparameters
├── data_preprocessing.py     # Dataset loading and cleaning
├── tokenization.py         # Tokenization utilities
├── model_training.py       # Model training logic
```

```
├── inference.py        # Translation inference
├── evaluation.py       # BLEU evaluation
├── main.py             # Main pipeline orchestrator
├── requirements.txt    # Python dependencies
└── README.md           # This file
```

## 🚀 Quick Start

### Installation

```bash
# Clone the repository
git clone <repository-url>
cd swahili-translation

# Install dependencies
pip install -r requirements.txt
```

### Usage

#### 1. Run Complete Pipeline (Train + Evaluate)

```bash
python main.py
```

This will:

- Load and preprocess the dataset
- Tokenize the data

- Fine-tune BLOOM model
- Evaluate with BLEU score
- Save the trained model

**2. Train Only**

```bash
python main.py --mode train
```

**3. Evaluate Existing Model**

```bash
python main.py --mode evaluate
```

**4. Interactive Translation**

```bash
python main.py --mode inference
```

Then enter Swahili sentences to translate:

```
Swahili: Habari yako?
English: How are you?
```

**Using Individual Modules**

**Data Preprocessing**

```python
from data_preprocessing import DataPreprocessor

preprocessor = DataPreprocessor()
train, val, test = preprocessor.process_all()
```

### Inference

```python
from inference import TranslationInference

inference = TranslationInference()
inference.load_model_and_tokenizer()
translation = inference.translate("Habari yako?")
print(translation)  # "How are you?"
```

### Evaluation

```python
from evaluation import TranslationEvaluator

evaluator = TranslationEvaluator()
evaluator.load_model()
results = evaluator.evaluate_bleu(test_dataset)
print(f"BLEU Score: {results['bleu_score']:.2f}")
```

## ⚙️ Configuration

Edit `config.py` to customize:

```python
class Config:
    MODEL_NAME = "bigscience/bloom-560m"
    BATCH_SIZE = 8
    LEARNING_RATE = 2e-5
    EPOCHS = 3
    MAX_LENGTH = 128
    # ... more parameters
```

## 📊 Dataset

- **Source**: Hugging Face `michsethowusu/english-swahili_sentence-pairs`
- **Size**: ~30,000 parallel sentences (after cleaning)
- **Split**: 80% train, 10% validation, 10% test

## 🎯 Performance

### Expected Results

- **Target BLEU Score**: > 25.0
- **Training Time**: 1-3 hours on GPU (much longer on CPU)
- **Memory**: ~4GB GPU RAM with FP16

### Improving Performance

1. **Increase epochs**: Try 5-7 epochs
2. **Use larger model**: BLOOM-1.1B or BLOOM-3B
3. **Tune hyperparameters**: Adjust learning rate, batch size
4. **Data augmentation**: Implement back-translation
5. **Ensemble**: Combine multiple model checkpoints

## 🔧 Advanced Usage

### Custom Training Arguments

```python
from model_training import TranslationModelTrainer

trainer = TranslationModelTrainer(tokenizer)
trainer.load_model()

# Modify training args before setup
trainer.config.EPOCHS = 5
trainer.config.LEARNING_RATE = 1e-5

trainer.setup_training(train_dataset, val_dataset)
trainer.train()
```

### Batch Translation

```python
```

```python
from inference import TranslationInference

inference = TranslationInference()
inference.load_model_and_tokenizer()

swahili_texts = ["Habari yako?", "Ninakupenda", "Asante sana"]
translations = inference.translate_batch(swahili_texts)
```

### Compare with Baseline

```python
from evaluation import TranslationEvaluator

evaluator = TranslationEvaluator()
evaluator.load_model()

# Provide baseline translations (e.g., from Google Translate)
baseline_translations = ["...", "...", "..."]
comparison = evaluator.compare_with_baseline(
    test_dataset,
    baseline_translations
)
```

## 📝 Command Line Options

```bash
```

```
# Run full pipeline
python main.py

# Run specific mode
python main.py --mode [train|evaluate|inference|full]

# Skip training (use existing model)
python main.py --skip-training

# Skip evaluation
python main.py --skip-evaluation
```

## 🐛 Troubleshooting

### Out of Memory Errors

```python
# In config.py, reduce batch size
BATCH_SIZE = 4  # or even 2
```

### Slow Training

```python
```

```
# Ensure GPU is being used
import torch
print(torch.cuda.is_available())  # Should be True

# Reduce number of training samples for testing
# In data_preprocessing.py, slice dataset
train_dataset = train_dataset.select(range(1000))
```

**Low BLEU Score**

- Train for more epochs (5-7)
- Increase model size (use BLOOM-1.1B)
- Check data quality
- Tune hyperparameters

## 📚 References

- BLOOM Model Paper
- Hugging Face Transformers
- SacreBLEU
- Dataset on Hugging Face

## 📄 License

This project is provided as-is for educational and research purposes.

## 🤝 Contributing

Contributions welcome! Areas for improvement:

- Multi-GPU training support
- Beam search optimization
- Alternative evaluation metrics (METEOR, ChrF)
- Web interface for translation
- Model quantization for faster inference

## 📧 Contact

For questions or issues, please open a GitHub issue.

---

**Built with ❤️ for low-resource language translation**